The C++ programming language

December 10, 2025

Ramasamy Kandasamy

Contents

1. Language

1.1. Fundamentals

Pre-processor

Pre-processor directive always start with a '#'.

- `#include` *filename*
- `#define NAME` *replacement text*
  `NAME` is replaced with *replacement text*
- `#define` *token*`(arg1,arg2)` *statement*
  Defining a macro. Eg:
  `#define max(A,B) ((A) > (B) ? (A) : (B))`
- `#undef NAME`
  Nullifies existing definition of `NAME`.
- `#if`, `#elif`,`#else`, `#endif`
- `#ifdef` and `#ifndef`
  Used in header-guards. Eg:

```
#ifndef HEADER_FILE_NAME
#define HEADER_FILE_NAME
...
#endif
```

- `#pragma once`: Single-inclusion header guard.
  Widely supported, but not guaranteed.
- `#error "message"`: Emit a compile-time error.
- `#warning "message"`: Emit a compile-time warning (compiler extension).
- `#line N "file"`: Change reported line/file in diagnostics.
- Built-ins: `__FILE__`, `__LINE__`, `__DATE__`, `__TIME__`, `__cplusplus(C++ version)`.

1.2. Expressions

Value Categories

`lvalue`	Has name. Can take address.
`prvalue`	Temporary value. No name. Guaranteed copy elision.
`xvalue`	Expiring. Eg: `std::move(x)`.
`glvalue`	`lvalue` ∪ `xvalue`.
`rvalue`	`prvalue` ∪ `xvalue`.
`L-value reference`	Refers to an `lvalue`. Eg: `int& ref = var;`
`R-value reference`	Refers to an `rvalue`. Eg: `int&& rref = 5;`

Exception: R-value can bind to const l-value reference.

```
const int& ref = 5;
```

Some exceptions:
- R-value can bind to const l-value reference.
  Eg: `const int& x = 42;`
- String literals are l-values.

Literals

`Boolean`	`true`	
	`false`	
`Integer`	`42`	Decimal.
	`4'200'000`	Decimal with separators.
	`0b101010`	Binary.
	`052`	Octal.
	`0x2a`	Hexadecimal.
`Float`	`3.14`	Double.
	`3.14f`	Float.
	`6.626e-34`	`xey` → $x \times 10^y$
`Character`	`'a'`	Ordinary literal.
	`'\n'`	Using escape seq.
	`'\13'`	
	`u8'a'`	UTF-8.
`String`	`"Hello"`	String literal.
`Pointer`	`nullptr`.	

NOTE: `Literals are prvalues.`
Exception: `String literals are lvalues.`

`std::initializer_list`

Constructed automatically from braced-init-list.
Eg:

```
Foo foo({1, 2, 3});
print_list({1, 2, 3});
```

Passed as argument to functions.
Eg:

```
void print\_list(std::initializer_list<int> list);
Foo(std::initializer_list<int> list);
```

Access elements using iterators or range-based for loop.

Arithemetic and logical operation

	Operators	Associativity
1	`() [] -> .`	left to right
2	`! ~ ++ -- + - * & (type) sizeof`	right to left
3	`* / %`(reminder)	left to right
4	`+ -`	left to right
5	`<< >>`	left to right
6	`< <= > >=`	left to right
7	`== !=`	left to right
8	`&`	left to right
9	`^`	left to right
10	`|`	left to right
11	`&&`	left to right
12	`||`	left to right
13	`?:`	left to right
14	`= += -= *= /= %= &= ^= |= <<= >>=`	right to left
15	`,`	left to right

Selected operators

`>>` Bitwise right shift operator.
`a = b >> 1` equivalent to dividing `b` by 2.

`<<` Bitwise left shift operator.
`a = b << 1` equivalent to multiplying `b` by 2.

`?:` Ternary conditional operator.
`x = (condition)? expr1: expr2;`
`expr1` → `condition == true`.
`expr2` → `condition == false`.

Type casting

`const_cast`	Modifies `const`/`volatile` qualifiers. Modifying a `const` object leads to UB.
`static_cast`	Compile-time downcast. Not safe.
`dynamic_cast`	Safe downcast. Eg: `Base* b = static_cast<Base*>d;` when invalid `b` is `nullptr`. Eg: `Base& b = static_cast<Base&>d;` when invalid throws `std::bad_cast`.
`reinterpret_cast`	

1.3. Declarations

<delc-type-specifier> <declarator>

Examples:

```
int x; // <int> <x>
int *x; // <int> <*x>
const int *x; // <const int> <*x>
int *const x; // <int> <const *x>
const int *const x; // <const int> <const *x>
```

Declarator

Tells the compiler about the name of the variable, its type (pointer, array, function etc).
Could also have cv qualifiers (const, volatile).
Eg: `int *const p;`
Here `const` is part of the declarator.

decl-type-specifiers

Fundamental types
`char`
`int`
`float`
`double`
`void`
Common aliases
`size_t`
`char_t`
`int8_t`
`int64_t`
`uint32_t`

NOTE: `std::size_t`, `int8_t` are preferred over `size_t`, `int8_t` etc. Why?

`register`	Ignored by modern compilers.
`mutable`	Removes const-ness from a const object member variable.
`inline`	Relaxes ODR. NOTE: Global linkage.
`static`	Specifies internal linkage. *Local:* initialized only once, retains value between calls. *Global:* visible only in the current TU. Different from `inline`. *See "Class" for static members.*
`extern`	Specifies external linkage. Eg: `extern int x; // x is now available in other TUs.`
`thread_local`	Specifies thread local storage duration. Eg: `thread_local int x;`
`typedef`	`typedef <type> <alias>;` Eg: `typedef long long int ll;`
`using`	`using <alias> = <type>;` Eg: `using ll = long long int;` `using` supports template aliase, but `typedef` does not.
`namespace`	Declares a namespace. Eg: `namespace Foo { ... }` Alias: `namespace Bar = Foo;`
auto	Type deduction based on initializer. Eg: `auto x = 10; // x is int`

decltype	Deduces type of an expression.
	Eg: `decltype(x) y; // y has same type as x`
	`decltype` is exact, unlike `auto`.
	Eg: `auto` doesn't preserve references, const, arrays decay to pointers etc.

`static_assert`	Compile time assertion.
	`static_assert(<bool-constexpr>)`
	`static_assert(<bool-constexpr>, "<message>")`

enum, enum class, union

enum Eg:

```
enum Color { RED,       // 0
             GREEN = 5, // 5
             BLUE       // 6
};
```

- No scope:Eg: `Color c = RED;`.
- Implicitly converts to int. Eg: `int x = RED;`.

enum class (scoped enum) Eg:

```
enum class Color {
    Red,        // 0
    Green = 3,  // 3
    Blue        // 4
};
```

Eg usage: `Color c = Color::Red;`.
Union
Eg:

```
union Data {
    int i;
    float f;
};
```

1.4. Initialization

Fundamental Types

```
int a;     // Garbage value.
int a{};   // a = 0.
int a{1};  // a = 1;
int a = b; // Copy intialization.
```

Array types

```
int a[3];          // Garbage value.
int a[3]{};        // a = {0, 0, 0}
int a{3}{1};       // a = {1, 0, 0}
int a{3}{1, 2, 3}; // a = {1, 2, 3}
```

Aggregate Types

Aggregates: Arrays, "Simple" structs/classes without Ctors.
Consider:

```
class Foo {
public:
    int x;
    double y;
};
```

```
Foo foo;
Foo foo{};      // foo.x = 0; foo.y = 0.0;
Foo foo{1};     // foo.x = 1; foo.y = 0.0;
Foo foo{1, 2};  // foo.x = 1; foo.y = 2.0;
```

Class Types with Constructors

```
Foo foo;         // Default Ctor.
Foo foo();       // Default Ctor.
Foo foo{};       // Default Ctor.
Foo foo{bar};    // Copy Ctor. Not blocked by explicity.
Foo foo(bar);    // Copy Ctor. Not blocked by explicity.
Foo foo = bar;   // Copy Ctor. Blocked if explicit.
Foo foo(1, 2);   // Parameterized Ctor.
Foo foo{1};      // Ctor with std::initializer_list.
Foo foo{1, 2};   // Ctor with std::initializer_list.
Foo foo({1, 2}); // Ctor with std::initializer_list.
```

1.5. Functions

- **Definition :** `T foo(args) {...}`.
- **Declaration :** `T foo(args);`.
- **Variadic Arguments:** `T foo(int n, ...)`.
  C-style. Not safe. Use variadic templates.

The main() function

```
main(int argc, char *argv[]) {
    ...
}
```

`argc`	No. of args including the program name.
`argv[0]`	Pointer to executable name.
`argv[i]`	Pointer to i^{th} argument.

Lambda function

Usage: `[<capture>](<params>)-><return-type>{<body>}`

`<capture>`	Specifics
`[]`	No variables captured.
`[=]`	All variables captured by value.
`[&]`	All variables captured by reference.
`[x, &y]`	x captured by value, y by reference.
`[=, &y]`	All variables except y by value, y by reference.

NOTE: Not specifying return type is not compile time error and ofter compiler can deduce it. But, it is good practice to specify it, especially for complex lambdas.
Eg:

```
// Sort descending using lambda
sort(nums.begin(), nums.end(), [](int a, int b) -> bool {
    return a > b;
});
```

Passing functions as arguments

1. Function pointers.
Eg:

```
int foo(int a, int b, int (*bar)(int, int)) {
    return bar(a, b);
}
// Function call.
foo(5, 3, add);
```

Doesn't work with lambdas with captures.

2. std::function

```
int foo(int a, int b, std::function<int(int, int)> bar) {
    return bar(a, b);
}
// Function call.
foo(5, 3, add);
```

3. Templates

```
template <typename bar>
int foo(int a, int b, bar bar) {
    return bar(a, b);
}
// Function call.
foo(5, 3, add);
```

Zero overhead

1.6. Statements

If-else	`if (`*expression*`)`
	`    `*statement*
	`else if (`*expression*`)`
	`    `*statement*
	⋮
	`else`
	`    `*statement*

Switch	`switch (`*expression*`)`
	`{`
	`    case `*const-expr*`:  `*statements*`; break;`
	`    case `*const-expr*`:  `*statements*`; break;`
	`    default:  `*statements*`; break;`
	`}`
	Without break statement the execution will *fall through* all the cases.
	`This can be used as:`
	`switch (`*expression*`)`
	`{`
	`    case 1:  case 2:  case 3:`
	`        group = 0;`
	`        break;`
	`    case 4:  case 5:`
	`        group = 1;`
	`        break;`
	`    default:`
	`        group = -1;`
	`        break;`
	`}`

While	`while (`*expression*`)` `    `*statement*

For	`for (expr1; expr2; expr3)`
	`    `*statement*
	This for loop is equivalent to:
	`expr1;`
	`while (expr2)`
	`{`
	`    statements;`
	`    expr3;`
	`{`

Do While	`do`
	`    `*statement*
	`while (`*expression*`);`

Break	`break` : Exit the the loop or control. `Works for while, for, do while and switch`

Continue	`continue`: Continue next iteration. Works for `while, for, do while` If `switch` is within a loop and if `continue` is used inside `switch` and if it is executed, then the loop skips to next loop.

Goto and label	Example:
	`if (disaster)`
	`{`
	`  goto error;`
	`}`
	...
	`error:`
	`{`
	`  statement`
	`}`

Shorter alternative for `if-else` statement:
`x = <cond-expr> ? <out-if-true> : <out-if-false>;`
Eg: `char is_pos = (n > 0) ? 'y' : 'n';`

1.7. Classes

Eg:

```
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
private:
    int x_;
    int y_;
};
```

Inheritance Behaviour

Simple rule

1. Private members are never inherited.
2. `derived-type = min(base-type, inheritance-type)`.

Member / Inheritance	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	NA	NA	NA

Members Variables

- `this` - pointer to the current object.
- `static` - shared across all objects of the class.

Member Functions

Rule-of-5 members

1. Copy Ctor

```
// Pass by ref to avoid recursion.
Vec::Vec(const Vec &other) {
    n_ = other.n_;
    data_ = new int[n_];
    for (int i = 0; i < n_; i++) {
        data_[i] = other.data_[i];
    }
}
```

2. Move Ctor

```
Vec::Vec(Vec &&other) noexcept {
    n_ = other.n_;
    data_ = other.data_;
    other.n_ = 0;
    other.data_ = nullptr;
}
```

3. Copy Assignment Operator

```
Vec& Vec::operator=(const Vec &other) {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = new int[n_];
        // Deep copy.
        for (int i = 0; i < n_; i++) {
            data_[i] = other.data_[i];
        }
    }
    return *this; // To permit chained assignments.
}
```

4. Move Assignment Operator

```
Vec& Vec::operator=(Vec &&other) noexcept {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = other.data_;
        other.n_ = 0;
        other.data_ = nullptr;
    }
    return *this; // To permit chained assignments.
}
```

5. Destructor

```
Vec::~Vec() {
    delete[] data_;
}
```

Others

6. Default Ctor Eg: `Foo::Foo()`
7. Converting Ctor
Ctor with single argument or multiple arguments with default values.

```
Say: Foo::Foo(const Bar&);

Foo f = bar; // Implicit conversion from Bar to Foo.

explicit Foo(const Bar&);// Prevents implicit
conversion.
```

8. Static methods
Belong to class, not object.
No `this` pointer.

Compilers provided default impelementation for 1-6 methods.

- `delete`: Prevents default generation.
- `default`: Makes the intent clear.
  Forces default generation?
- `explicit`: Prevents implicit conversions for converting ctors.

Virtual Methods and Abstract Class

- **Virtual method:** Eg: `virtual void Animal::speak();`
- **Pure virtual method:** Eg: `virtual void Animal::speak() = 0;`
- **Abstract class:** Class with at least one pure virtual method. Cannot instantiate objects of abstract class.

Virtual vs normal methods:

```
Animal* a = new Dog();
a->speak();
```

- `speak` is virtual: Dog's speak() called.
- `speak` is normal: Animal's speak() called.

override & final

Specifiers at the end. Relative order between them doesn't matter.
`override`:
Tells a method is meant to override a base class method.
Compiler error if it doesn't.
Eg:

Base: `void Animal::speak() const;`
Derived: `void Dog::speak(); // Compiles.`
Unexpected runtime behaviour:
`a->speak(); // Base class speak.`
Derived w/ override:
`void Dog::speak() override; // Compiler error.`

`final`:
Method not supposed to be overridden in derived classes.
Compiler error if it is.
Eg:

Base: `void Dog::speak() override final;`
Derived:
`void BorderCollie::speak() override; // Compiler error.`

1.8. Templates

Template Parameter Kinds

Type	`typename T` or `class T` Accepts any type as argument.
Non-type	`int N`, `size_t N`, `auto N`, etc. Accepts compile-time constant values. Types: integral, pointers, references, `nullptr_t`, enums, float (C++20).
Template template	`template <typename> class C` `template <typename> typename C` (C++17+) Accepts template as argument.

Template Parameter Syntax

Basic	`typename T`
	`int N`
	`template <typename> class C`

With default	`typename T = int`
	`int N = 10`
	`template <typename> class C = std::vector`

Variadic	`typename... Args`
	`int... Ns`
	`template <typename> class... Containers`

NOTE: `typename` vs `class` for type parameters are interchangeable. `typename` is preferred (more explicit). Also, `typename` has another use: disambiguating dependent types. Eg: `typename T::value_type x;`

Template Specialization

Full/Explicit Specialization
Provide custom implementation for specific type(s).

Eg:

```
// Primary template
template <typename T>
class MyClass { /*...*/ };

// Full specialization for bool
template <>
class MyClass<bool> { /*...*/ };
```

Partial Specialization
Specialize for a subset of types. Only for class templates, not functions.

Eg:

```
// Primary template
template <typename T, typename U>
class Pair { /*...*/ };

// Partial specialization: both types same
template <typename T>
class Pair<T, T> { /*...*/ };

// Partial specialization: pointer types
template <typename T, typename U>
class Pair<T*, U*> { /*...*/ };
```

Full specialization	`template <> class C<int> {...}` All parameters specified. Works for classes and functions.
Partial specialization	`template <typename T> class C<T*> {...}` Some parameters remain generic. Only for class templates.

Parameter Packs

Syntax

Declaration	`typename... Args` `int... Ns`

Expansion	`Args...` expands to `T1, T2, T3, ...`
	`func(Args...)...` expands to `func(T1), func(T2), ...`
	`sizeof...(Args)` gives pack size.

Example: Compile-time Fibonacci sequence

```
// Fibonacci calculation
template <int N>
struct Fib {
    static constexpr int value =
        Fib<N-1>::value + Fib<N-2>::value;
};
template <>
struct Fib<0> { static constexpr int value = 0; };
template <>
struct Fib<1> { static constexpr int value = 1; };

// Variadic template to hold sequence
template <int... Ns>
struct FibSeq {
    static constexpr int values[] = { Fib<Ns>::value... };
};

// Usage
constexpr int fibs[] = FibSeq<0, 1, 2, 3, 4, 5>::values;
// fibs = {0, 1, 1, 2, 3, 5}
```

Pack expansion `Fib<Ns>::value...` expands to
`Fib<0>::value, Fib<1>::value, ..., Fib<5>::value`

1.9. Miscellaneous

2. Utils

2.1. Data Elements

Pair

Eg: `pair<int, int> x = 1,2;`

`x.first`	Returns 1.
`x.second`	Returns 2.

Tuple

Fixed length collection of heterogenous values.
Eg: `tuple<int, string, double> t;`

Operations related to tuple

`get<i>`	Return i^th value of t. Eg: `get<0>(t);`.
`make_tuple()`	Returns tuple out of individual values. Eg: `make_tuple(5, "Paul", 23.4)`.
`tie()`	Unpacking tuple. Eg: `tie(n, s, x) = t;` `tie(n, std::ignore, x) = t;`

Bit set

Definiton

`bitset<size> s;`
Examples:
`bitset<5> s; bitset<5> s(string("1100110"));`

Operations on Bit set:

`a & b`	Bitwise AND.
`a | b`	Bitwise OR.
`a ^ b`	Bitwise EXOR.
`a`	Bitwise negation.

2.2. Memory management in C++

- `void* operator new(size)`
  Eg: `myClass* p1 = new myClass;`
  `myClass* p1 = new(sizeof(myClass));`
- `void delete(void* ptr)`
  Eg: `delete p1; delete(p1);`

2.3. limits

Examples:

- `numeric_limits<int>::min()`
- `numeric_limits<int>::max()`
- `numeric_limits<double>::min()`
- `numeric_limits<double>::infinity()`

3. Iterators

4. Containers

4.1. Sequential Containers

Vectors

Definiton:

`vector<T> v;`

Element access

`operator[]`	
`v.back()`	Last element
`v.front()`	First element

Modifiers

`v.push_back(5)`	Add `5` to the vector. Simillar to push in stack data structure.
`v.insert()`	Inserts elements / range in `v`. Eg: Append `u` to `v`. `v.insert(v.end(), u.begin(), u.end());`
`v.emplace`	`v.emplace{iterator pos, Args}`. New element is constructed in place using `Args` and inserted at pos.
`v.emplace_back`	`v.emplace_back{Args}`. New element is constructed in place using `Args` and inserted at the end. IMPORTANT: Note the differences between pushback, insert, emplace and emplaceback.
`v.pop_back()`	Remove element on top of the stack.
`v.erase()`	Remove element / range from `v`.
`v.clear()`	Clears all the elements in `v`.

Capacity

`v.size()`	Returns the size of the vector.
`v.capacity()`	Current memory allocated to `v` in terms of number of elements.
`v.max_size()`	Maximum memory available for `v`.
`v.reserve()`	Reserve a minimum size for v. Does not affect v.size().
`v.shrink_to_fit()`	Shrink the capacity to fit the size.
`v.empty()`	Test if `v` is empty.

`tie` Can be used to unpack a vector.
Example: `tie(a, b, c) = v;`

Queue

Eg: `queue<int> q;`

`q.push(5)`	Adds element to the end.
`q.pop()`	Removes first element.
`q.front()`	Returns first element.
`q.empty()`	True if q is empty.

Dequeue

Eg: `dequeue<int> d;`

`d.push_back(5)`	Add to the top.
`d.push_front(2)`	Add to the bottom.
`d.pop_back()`	Remove from the top.
`d.pop_front()`	Remove from the bottom.

4.2. Associative Containers

Set

Definition

```
set<type> s;
multiset<type> s;
unordered_set<type> s;
unordered_multiset<type, hasher> s;
```

`hasher`: Refer Custom Hash Function.

Set operations

`s.insert(item);`	
`s.erase(item);`	
`s.count(item);`	
`s.find(item);`	Returns the iterator that points to `item`.
`s.insert()`	Inserts elements in s. Eg: Insert elements from t to s. `v.insert(v.end(), u.begin(), u.end());`
`s.size()`	Not availabe for `unordered_set` and `unordered_multiset`.
`s.lower_bound(x)`	Returns iterator to the smallest element that is larger than or equal to x.
`s.upper_bound(x)`	Returns iterator to the smallest element that is larger than x.

NOTE: erase method remove all instances of the item in the multiset.

Map

`map` : Balanced binary tree.
`unordered_map`: Hash list.

Definition:

```
map<type1, type2> m;
```

Eg: `map<string, int> m;`

```
unordered_map<type1, type2, hasher> m;
```

`hasher`: Refer Custom Hash Function.

Operations on map

`m["banana"]`	Retrive value associated with `banana`.
`m.count("banana")`	Return 1 if the key is present else 0.
`m.size();`	

4.3. Container Adapters

Stack

Eg: `stack<int> s;`
s.push(5)
s.top()
s.pop()

Priority queue

Format:
`priority_queue(type, vector<type>, compare)`
Max priority queue [Default]
Eg: `priority_queue<int> q;`
Min priority queue
`priority_queue<int, vector<int>, greater<int>> q;`

`q.push(3)`	
`q.top()`	Returns the largest element.
`q.pop()`	Removes the largest element.

5. Algorithms

Sort and search

Search and sort from C

- `void *bsearch(const void *key, const void *base, size_t n, size_t size, int (*cmp) (const void *keyal, const void *datum))`
  Searches `base[0]` $\cdots$ `base[n-1]` for `key`.
  Comparison function must return negative if its first argument (search key) is less than its second argument (a table entry) and so on.
- `qsort(void *base, size_t n, size_t size, int (*cmp)(const *void, const *void))`

Search and sort from C++

Compare functions:

```
bool compare(x, y){
    return(x strictly precedes y)
}
```

In all the below, a could be an iterator or a pointer and all of them can use custom compare functions.

- y = `lower_bound(a, a+n, x)`
  Returns an iterator to the first element whose value is $\geq$ x.
- y = `upper_bound(a, a+n, x)`
  Returns an iterator to the first element whose value is $>$ x.
- y = `equal_range(a, a+n, x)`
  Returns a pair of iterators pointing to the upper bound and lower bound of x.
- y = `equal_range(a, a+n, x, compare_function)`
  Returns a pair of iterators pointing to the upper bound and lower bound of x.
- `sort(it1, it2, compare_function);`
- `stable_sort(it1, it2, compare_function);`
- `is_sorted(it1, it2, compare_function);`
- `is_sorted_until(it1, it2, compare_function);`
  Returns the iterator to the element until which the array is sorted.
- `find_if(start_it, end_it, func)`
  Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns true.
- `find_if_not(start_it, end_it, func)`
  Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns false.

Min Max

- `max(a, b, comp); min(a, b, comp);`
- `max({a1, a2, a3}, comp); min({a1, a2, a3}, comp);`
- `max_element(a, a+n, comp);`
  Returns the iterator for the largest element in a. If there are multiple largest element, returns the iterator/pointer for the first one.
- `min_element(a, a+n, comp);`

Merge

- `merge(InputIterator F1, InputIterator L1, InputIterator F2, InputIterator L2, OutputIterator R);`
  Returns the iterator to the end of output.
  NOTE: This operator does not allocate memory. The `OutputIterator R` should point to a vector with sufficient memory to store the concatenated vector.
- `set_intersection` Arguments and output are the same as that for `merge`.
- `set_union` Arguments and output are the same as that for `merge`.
- `set_difference` Arguments and output are the same as that for `merge`.
- `set_symmetric_difference` Arguments and output are the same as that for `merge`.

Hash Function

Can be used for custom types in `unordered_map` and `unordered_set`.

Usage:

```
hash<T> hasher;
size_t hasvalue = hasher(x);//Where x is an object of
type T
```

Eg:

```
hash<int> h; size_t x = h(10);
hash<string> h; size_t x = h("hello");
```

Custom hash function.
Custom hash function for a user defined type can be defined as follows:

```
class CustomHash {
 public:
 size_t operator () (const pair<string, string>& p) const {
     hash<string> hasher;
     auto h1 = hasher(p.first);
     auto h2 = hasher(p.second);
     return h1 ^ h2; // Below for better heuristics.
 }
};
```

Heuristics to combine hash values.
`h2 = h2 ^(h1 << 1); h2 ^= h2 + 0x9e3779b9 + (h1<<6) + (h1>>2);`
0x9e3779b9 is a prime number.
See: https://stackoverflow.com/a/2595226/5607735

Others

- `swap(T& a, T&b);`
- `reverse(Iterator first, Iterator last);`
- `next_permutation(a.begin(), a.end());`
- `prev_permutation(a.begin(), a.end());`
- `random_shuffle(a.begin(), a.end());`
  Gives the next permutation for a given vector of integers.

6. Libraries

6.1. Input output

File Access

Format:
```
FILE *fp;
FILE *fopen(char *name, char *mode);
int fclose(FILE *fp);
fp = fopen(name, mode);
```
Allowable modes include:

`"r"`	read.
`"w"`	write.
`"a"`	append.
`"b"`	binary files: `"rb"`, `"wb"`, `"ab"`.

NOTE: Difference between `rb` and `r` etc:
While using just `"r"` might work in writing binary files, it might cause trouble due misinterpretation of special characters like LF and CR.
See: https://stackoverflow.com/q/2174889/5607735
Standard IO Can be treated in the same way as file pointer.
`stdin`, `stdout`, `stderr`

Reading and writing a file

```
size_t fread{const void* ptr, size_t size,/
size_t count, FILE* stream};

size_ t fwrite{const void* ptr, size_t size,/
size_t count, FILE* stream};
```

Character I/O

`getchar`	Takes input from standard input. `int getchar()` Equivalent to `getc(stdin)`
`putchar`	Output to standard output. `int putchar(int)` Equivalent to `putc(c), stdout`
`getc`	`int getc(FILE *fp)`, Eg: `getc(stdin)`
`putc`	`int putc(int c, FILE *fp)`
`ungetc`	

Line I/O

`gets`	Reads until `EOF`. `gets` deletes terminal `\n`
`puts`	Writes line to stdout. `puts` adds terminal `\n`
`fgets`	`char *fgets(char *line, int maxline, FILE *fp)`
`fputs`	`int *fputs(char *line, FILE *fp)`
`getline`	Eg: `getline(cin, s)`. Where `s` is a string.

NOTE: Never use `gets`
`gets` does not check for buffer overrun, and keeps reading until it encouters new line or `EOF`. It has been used to break computer security.

`printf` and `scanf`

Conversion formats for `printf` and `scanf`

Character	Argument type; Printed as
`d,i`	int; decimal number
`o`	int; unsigned octal number (without leading zero)
`x,X`	int: unsigned hexadecimal number (without a leading 0x or 0X), using `abcdef` or `ABCDEF`.
`u`	int; unsigned decimal number.
`zu`	`size_t`.
`c`	int; single character.
`s`	char *; Print string, scan word. NOTE: `scanf` reads only a word and not the entire string.
`f`	double; [-]$m.dddddd$.
`e,E`	double; [-]$m.dddddd \pm xx$, or [-]$m.dddddd \pm xx$.
`g,G`	double; %e or %E if exponent is < -4 or >= precision, else use %f.
`p`	void *; pointer.
`%`	no argument; Print %.

Between % and conversion character there may be in order:

- Minus sign: Left justification.
- Number: Minimum field width.
- Period: Separates field width from precision.
- Number: Precision.
  For float: number of digits after decimal.
  For int: minimum number of digits.
  For string: maximum number of characters to be printed.
- h: for short integer. l: for long integer.

Formatting rules

Examples

	Wildcard
%*d	Here precision can be specified dynamically. Eg: `printf("%*d", 6, foo);`, this is equivalent to `printf("%6d",foo);`
	Integers
%d	print as decimal integer.
%6d	print as decimal integer, at least 6 characters wide.
	Floating point numbers
%6f	print as floating point.
%.2f	print as floating point, 2 characters after decimal point.
%6.2f	print as floating point, at least 6 characters wide and 2 characters after decimal point.
	String: example `hello, world` (12 chars).
`:%s:`	`:hello,␣world:`
`:%10s:`	`:hello,␣world:`
`:%.10s:`	`:hello,␣wor:`
`:%-10s:`	`:hello,␣world:`
`:%.15s:`	`:hello,␣world:`
`:%-15s:`	`:hello, world␣␣␣:`
`:%15 .10s:`	`:␣␣␣␣␣hello, wor:`
`:%-15 .10s:`	`:hello,␣wor␣␣␣␣:`

Printf and variants

- `int printf(char *format, arg1, arg2, ···);`
  Print output to stdout.
- `int sprintf(char *string, char *format arg1, arg2, ···);`
  Print output to `string`.
- `int fprintf(FILE *fp, char *format arg1, arg2, ···);`
  Print output to file pointed by the file pointer `fp`.

Example:

```
printf("%s\n", string_var);
printf("%s", "hello, world");
printf("square of n is %d\n", n_square);
```

Scanf and variants

- `int scanf(char *format, arg1, arg2, ···);`
  Scan input from `stdin`.
- `int sscanf(char *string, char *format arg1, arg2, ···);`
  Scan input from string.
- `int fscanf(FILE *fp, char *format arg1, arg2, ···);`
  Scan input from the file pointed by the file pointer `fp`.

NOTE1: unlike printf, in scanf the variable are pointers.
NOTE2: In `scanf %s` reads a word and not the entire string.
Also see: https://stackoverflow.com/a/1248017/5607735

Examples:

```
scanf("%d", &n);
sscanf(string, "%d", &n);
fscanf(fp, "%d", &n);
```

6.2. `cin` and `cout`

`cin`	Read input until space, or tab or LF.
`cout`	Output.

NOTE:
• Use the following to increase speed ot I/O.
`ios_base::sync_with_stdio(false);`
`cin.tie(nullptr); cout.tie(nullptr);`
• Use `cin.ignore()` after `cin` and before using `getline`.

6.3. Miscellaneous

`int fflush(FILE *stream)`
Causes any buffered but unwritten data to be written. The effect is undefined on an input stream.
`int fclose(FILE *stream)`
Flushes any unwritten data for `stream`, discardes any unread buffered input.

6.4. String

A `string` is like a vector of characters.

Definition:
`string s = "Hello";`

String operations:

`a + b`	Concatenate.
`s.append(t)`	append `t` to `s`.
`s.compare(t)`	returns `0` if `s == t`, returns < 0 if $s \prec t$, else returns > 0.
`s.substr(start, len)`	Select a substring.
`s.find(str2)`	find first occurence of `str2`.
`s.find(str2, pos)`	Find first occurence of `str2` by searching from the positon `pos`.

7. Devtools

7.1. G++

Usage:
`g++ option hello.cpp -o hello`
`g++ hello.cpp -o hello`
`g++ -O3 hello.cpp -o hello`
Options:

`-o`	Output file name.
`-I`	Include additional directories in the search path. Eg: `g++ -I /home/user/libs foo.cpp`
`-O<level>`	Optimization level, starts from 0. Eg: `-O3`
`-g`	Creates breakpoints for debugging using GDB.
`-lm`	Missing link. Used to link certain libraries like `math.h`.
`-E`	Output preprocessed but uncompiled code.
`-S`	Compile but do not assemble.
`-c`	Compile and assembly, but do not link.

7.2. GDB

To run: `gdb hello`

`break`	Set break point. Eg: `break function` or `break line number`.
`run`	Run the program. The program stops at every break point.
`next`	Run until next breakpoint.
`print`	Eg: `print i` or `print &i`
`sizeof`	Eg: `print sizeof(i)`
`&i`	Address of i.
`*j`	Content of memory location j.
`ptype`	get type of a variable. Eg: `ptype(i)`
`set var`	Reassign variable. Eg: `set var i = 1`
`disassemble`	Use after break and run. Gives assembly code.

Accessing memory location using x
Usage: `x/nfs`. `nfs` describes the format.
n - Number of units to display.
f - Number format.
s - Size of each unit.

`x`	Hex.
`o`	Octal.
`t`	Binary.
`d`	Decimal.
`u`	Unsigned decimal.
`i`	Instruction.
`c`	Character.
`s`	String.
`b`	byte.
`h`	Halfword.
`w`	Word.
`g`	Giant.